Propositional Logic Formula Parser and Binary Tree Builder

Fatemeh Abshang

Spring 2024–2025

Abstract

This document presents a Python implementation of a propositional logic formula parser that constructs binary parse trees from logical expressions. The system verifies whether a given formula is well-formed (WFF) and generates a hierarchical tree representation based on operator precedence. The parser handles standard logical operators including negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), and implication ($\rightarrow$).

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 Logical Operators . . . 3

2 System Architecture **3**
2.1 Data Structures . . . 3
2.1.1 Node Class . . . 3
2.2 Key Components . . . 4
2.2.1 Input Processing . . . 4
2.2.2 Parentheses Validation . . . 4

3 Core Algorithm: Binary Tree Construction **5**
3.1 Algorithm Overview . . . 5
3.2 Detailed Algorithm . . . 5
3.3 Removing Outer Parentheses . . . 5

4 Well-Formed Formula (WFF) Validation **5**
4.1 Definition . . . 5
4.2 Validation Algorithm . . . 7

5 Examples **7**
5.1 Example 1: Simple Formula . . . 7
5.2 Example 2: Complex Formula . . . 7
5.3 Example 3: Negation . . . 8
5.4 Example 4: Precedence . . . 8

6 Complexity Analysis **8**
6.1 Time Complexity 8
6.2 Space Complexity 8

7 Usage Instructions **9**
7.1 Input Format 9
7.2 Running the Program 9

8 Limitations and Future Work **10**
8.1 Current Limitations 10
8.2 Potential Enhancements 10

9 Conclusion **10**

A Complete Source Code **10**

1 Introduction

1.1 Project Overview

Propositional logic is a fundamental branch of mathematical logic dealing with propositions and their logical relationships. This project implements a formula parser that:

- Reads propositional logic formulas from text files
- Validates formula syntax and structure
- Constructs binary parse trees based on operator precedence
- Verifies well-formedness of formulas (WFF)
- Provides visual tree representation

1.2 Logical Operators

The parser supports four standard logical operators with the following precedence hierarchy:

Operator	Symbol	Python Code	Precedence
Implication	$\rightarrow$	`IMP`	1 (lowest)
Disjunction	$\vee$	`OR`	2
Conjunction	$\wedge$	`AND`	3
Negation	$\neg$	`NOT`	4 (highest)

Table 1: Logical operators and their precedence levels

Lower precedence values indicate that an operator binds less tightly, meaning it is evaluated later in the parsing process.

2 System Architecture

2.1 Data Structures

2.1.1 Node Class

The binary tree is constructed using a simple node structure:

```python
class Node:
    def __init__(self, key):
        self.data = key
        self.left: Node | None = None
        self.right: Node | None = None
```

Listing 1: Node class definition

Each node contains:

- `data`: The logical operator or propositional variable

- `left`: Pointer to the left child (left operand)
- `right`: Pointer to the right child (right operand)

For binary operators ($\wedge, \vee, \rightarrow$), both children exist. For negation ($\neg$), only the right child exists. Leaf nodes (propositional variables) have no children.

2.2 Key Components

2.2.1 Input Processing

The `get_input()` function reads formulas from files and preprocesses them:

```python
def get_input():
    print("Please enter a file name:")
    file_name = input()
    with open(file_name, 'r', encoding='utf-8') as file:
        formula = file.read().strip()
        formula = formula.replace(" ", "")
        return formula
```

Listing 2: Input processing function

Key operations:

1. Prompt user for filename
2. Read file contents with UTF-8 encoding
3. Remove whitespace to simplify parsing

2.2.2 Parentheses Validation

The `check_parentheses()` function ensures balanced parentheses using a stack:

Algorithm 1 Parentheses Validation

```
function CheckParentheses(formula)
    stack ← []
    for char in formula do
        if char = '(' then
            stack.push(char)
        else if char = ')' then
            if stack is empty then
                return false
            end if
            stack.pop()
        end if
    end for
    return stack is empty
end function
```

3 Core Algorithm: Binary Tree Construction

3.1 Algorithm Overview

The `binary_tree()` function recursively builds a parse tree from a formula string. It follows these steps:

1. **Base Case**: Single variable → create leaf node
2. **Outer Parentheses**: Remove and recurse on inner content
3. **Find Main Operator**: Identify lowest-precedence operator outside parentheses
4. **Build Subtrees**: Recursively construct left and right children
5. **Handle Negation**: Special case for unary operator

3.2 Detailed Algorithm

3.3 Removing Outer Parentheses

The `remove_outer_parentheses()` function checks if the entire formula is enclosed in matching parentheses:

```python
def remove_outer_parentheses(formula, begin, end):
    if formula[begin] == '(' and formula[end] == ')':
        count = 0
        for i in range(begin, end + 1):
            if formula[i] == '(':
                count += 1
            elif formula[i] == ')':
                count -= 1
            if count == 0 and i < end:
                return False
        return True
    return False
```

Listing 3: Outer parentheses removal

This ensures we only remove parentheses that encompass the entire subformula, not just a portion.

4 Well-Formed Formula (WFF) Validation

4.1 Definition

A well-formed formula in propositional logic must satisfy:

- Propositional variables are lowercase letters
- Negation ($\neg$) is a unary operator with one operand
- Binary operators ($\wedge, \vee, \rightarrow$) have exactly two operands
- The structure follows proper syntax rules recursively

Algorithm 2 Binary Tree Construction

```
function BinaryTree(formula, begin, end)
    if begin > end then
        return null
    end if
    if begin = end and formula[begin] is lowercase letter then
        return new Node(formula[begin])
    end if
    if RemoveOuterParentheses(formula, begin, end) then
        return BinaryTree(formula, begin + 1, end − 1)
    end if
    mainOperator ← −1
    minPrecedence ← 5
    parenthesesDepth ← 0
    for i ← begin to end do
        if formula[i] = '(' then
            parenthesesDepth ← parenthesesDepth + 1
        else if formula[i] = ')' then
            parenthesesDepth ← parenthesesDepth − 1
        else if parenthesesDepth = 0 and formula[i] is binary operator then
            precedence ← OPERATOR_PRECEDENCE[formula[i]]
            if precedence < minPrecedence then
                minPrecedence ← precedence
                mainOperator ← i
            end if
        end if
    end for
    if mainOperator ≠ −1 then
        node ← new Node(formula[mainOperator])
        node.right ← BinaryTree(formula, mainOperator + 1, end)
        node.left ← BinaryTree(formula, begin, mainOperator − 1)
        return node
    end if
    if formula[begin] = ¬ then
        node ← new Node(¬)
        node.right ← BinaryTree(formula, begin + 1, end)
        return node
    end if
    return null
end function
```

4.2 Validation Algorithm

The `is_WFF()` function recursively validates the parse tree:

Algorithm 3 WFF Validation

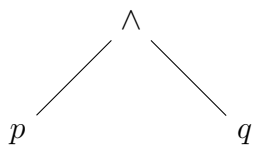
```
function IsWFF(node)
    if node = null then
        return false
    end if
    if node.left = null and node.right = null then
        return node.data is lowercase letter
    end if
    if node.data = ¬ then
        return node.left = null and IsWFF(node.right)
    end if
    if node.data ∈ {∧, ∨, →} then
        return node.left ≠ null and node.right ≠ null
              and IsWFF(node.left) and IsWFF(node.right)
    end if
    return false
end function
```

5 Examples

5.1 Example 1: Simple Formula

Input: $(p \wedge q)$

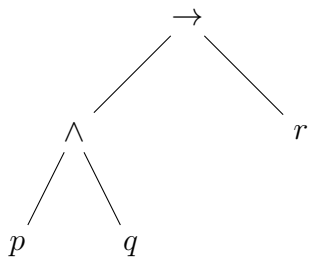
Parse Tree:



5.2 Example 2: Complex Formula

Input: $((p \wedge q) \rightarrow r)$

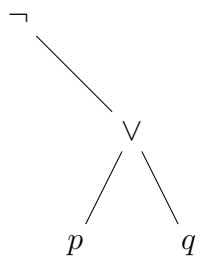
Parse Tree:



5.3 Example 3: Negation

Input: $\neg(p \vee q)$

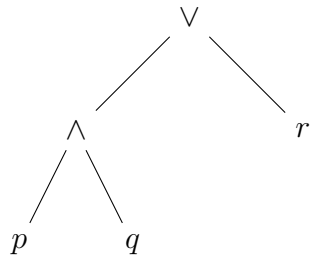
Parse Tree:



5.4 Example 4: Precedence

Input: $p \wedge q \vee r$ (assuming implicit parentheses)

Due to precedence ($\wedge$ has higher precedence than $\vee$), this is parsed as $(p \wedge q) \vee r$:



6 Complexity Analysis

6.1 Time Complexity

- **Parentheses Check**: $O(n)$ where n is formula length
- **Binary Tree Construction**: $O(n^2)$ worst case
  - Each recursive call scans the substring: $O(n)$
  - Maximum recursion depth: $O(n)$
  - Total: $O(n^2)$
- **WFF Validation**: $O(n)$ - single tree traversal

6.2 Space Complexity

- **Tree Storage**: $O(n)$ - one node per operator/variable
- **Recursion Stack**: $O(h)$ where h is tree height
- Worst case: $O(n)$ for unbalanced trees

7 Usage Instructions

7.1 Input Format

Create a text file containing a propositional logic formula:

- Use lowercase letters for propositional variables: `p, q, r, s, ...`
- Use the following symbols for operators:
  - Negation: $\neg$ (Unicode: U+00AC)
  - Conjunction: $\wedge$ (Unicode: U+2227)
  - Disjunction: $\vee$ (Unicode: U+2228)
  - Implication: $\rightarrow$ (Unicode: U+2192)
- Use parentheses `()` for grouping
- Spaces are ignored

Example file content:

```
((p ^ q) -> (r v ~s))
```

Where:

- `~` represents $\neg$
- `^` represents $\wedge$
- `v` represents $\vee$
- `->` represents $\rightarrow$

7.2 Running the Program

```
# Run the program
python formula_parser.py

# Enter filename when prompted
Please enter a file name:
formula.txt

# Output will show validity and tree structure
Valid formula!
→
  ∧
    p
    q
  ∨
    r
    ¬
      s
```

Listing 4: Execution steps

8 Limitations and Future Work

8.1 Current Limitations

1. No support for equivalence ($\leftrightarrow$) or XOR operators
2. Variables limited to single lowercase letters
3. No error recovery for malformed formulas
4. Basic text-based tree visualization

8.2 Potential Enhancements

1. **Extended operators**: Add equivalence, XOR, NAND, NOR
2. **Multi-character variables**: Support `p1, var2, etc.`
3. **Truth table generation**: Evaluate formulas for all variable assignments
4. **Formula simplification**: Apply logical equivalences
5. **Graphical visualization**: Generate tree diagrams using GraphViz
6. **Syntax error messages**: Provide detailed error locations
7. **Interactive mode**: Allow formula editing and real-time validation

9 Conclusion

This propositional logic parser provides a solid foundation for working with logical formulas. The binary tree representation enables efficient formula manipulation and validation. The modular design allows for easy extension with additional operators and features.

The parser correctly handles operator precedence, validates formula syntax, and provides a clear tree representation. It serves as both an educational tool for understanding propositional logic structure and a practical utility for logical formula processing.

A Complete Source Code

```
# Define logical symbols used in formula
NOT = '¬'
AND = '∧'
OR = '∨'
IMP = '→'

## operator precedence
OPERATOR_PRECEDENCE = {
    IMP: 1, OR: 2, AND: 3, NOT: 4
}

# get formula from user file input and remove spaces
```

```python
def get_input():
    print("Please enter a file name:")
    file_name = input()
    with open(file_name, 'r', encoding='utf-8') as file:
        formula = file.read().strip()
        formula = formula.replace(" ", "")
        return formula

# define the class for the nodes
class Node:
    def __init__(self, key):
        self.data = key
        self.left: Node | None = None
        self.right: Node | None = None

# check balanced parentheses using a stack
def check_parentheses(formula):
    stack = []
    for char in formula:
        if char == '(':
            stack.append(char)
        elif char == ')':
            if not stack:
                return False
            stack.pop()
    return not stack

# check if the formula is a well-formed formula
def is_WFF(node):
    if node is None:
        return False

    if node.right is None and node.left is None:
        return node.data.isalpha() and node.data.islower()

    if node.data == NOT:
        return node.left is None and is_WFF(node.right)

    if node.data in {AND, OR, IMP}:
        return (node.left is not None and
                node.right is not None and
                is_WFF(node.left) and
                is_WFF(node.right))

    return False

# remove the outer parentheses
def remove_outer_parentheses(formula, begin, end):
    if formula[begin] == '(' and formula[end] == ')':
        count = 0
        for i in range(begin, end + 1):
```

```
            if formula[i] == '(':
                count += 1
            elif formula[i] == ')':
                count -= 1
            if count == 0 and i < end:
                return False
        return True
    return False

# recursively builds a binary parse tree
def binary_tree(formula, begin, end):
    if begin > end:
        return None

    if (begin == end and formula[begin].isalpha()
        and formula[begin].islower()):
        node = Node(formula[begin])
        return node

    if remove_outer_parentheses(formula, begin, end):
        return binary_tree(formula, begin + 1, end - 1)

    main_operator = -1
    min_value = 5
    parentheses_check = 0

    for i in range(begin, end + 1):
        if formula[i] == '(':
            parentheses_check += 1
        elif formula[i] == ')':
            parentheses_check -= 1
        elif parentheses_check == 0:
            if formula[i] in [IMP, OR, AND]:
                flag = OPERATOR_PRECEDENCE[formula[i]]
                if flag < min_value:
                    min_value = flag
                    main_operator = i
                elif flag == min_value:
                    if formula[i] in [AND, OR]:
                        min_value = i

    if main_operator != -1:
        char_main_operator = formula[main_operator]
        node = Node(char_main_operator)
        node.right = binary_tree(formula, main_operator+1, end)
        node.left = binary_tree(formula, begin, main_operator-1)
        return node

    if formula[begin] == NOT:
        node = Node(NOT)
        node.right = binary_tree(formula, begin + 1, end)
```

```python
        return node

    return None

# print the parse tree
def print_tree(node, level=0):
    if node is None:
        return
    print("  " * level + str(node.data))
    print_tree(node.left, level + 1)
    print_tree(node.right, level + 1)

# main function
def main():
    formula = get_input()
    if check_parentheses(formula):
        tree = binary_tree(formula, 0, len(formula) - 1)
        if tree is not None and is_WFF(tree):
            print("Valid formula!")
            print_tree(tree)
        else:
            print("Invalid formula!")
    else:
        print("Invalid formula!")

if __name__ == "__main__":
    main()
```

Listing 5: Full implementation